

DOCUMENTACION TECNICA

Runbook: puente Hermes

ESP32-S3 Zero

[RUNBOOK-HERMES-BRIDGE.MD](#)

Runbook: puente Hermes — ESP32-S3 Zero

TABLA DE CONTENIDOS

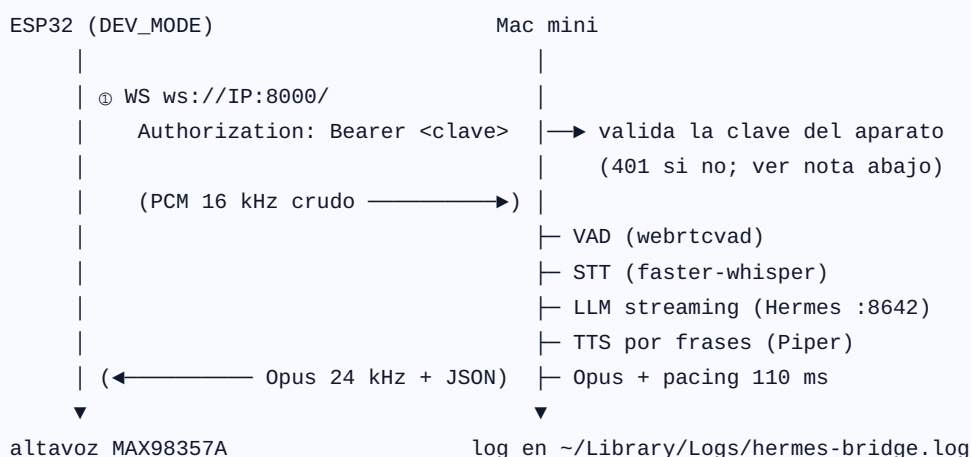
Runbook: puente Hermes — ESP32-S3 Zero	2
Descripcion general.....	2
Flujo de operacion final	3
Requisitos previos	3
Paso 1 — Habilitar la API de Hermes	3
Paso 2 — Fijar la IP de la Mac mini	4
Paso 3 — Instalar el puente	4
Paso 4 — Descargar la voz de Piper (español)	5
Paso 5 — Probar el puente manualmente	5
Variables de configuracion (todas opcionales)	7
Paso 6 — Firmware: IP y flasheo	8
Paso 7 — Verificacion end-to-end.....	8
Paso 8 — Dejarlo como servicio (launchd)	9
VoiceHistory y contexto Telegram	9
Rollback a la nube de Elato.....	10
Solucion de problemas.....	11
Referencia rapida	13

Descripcion general

Procedimiento completo para poner en marcha el **puente ElatoAI ↔ Hermes**: el ESP32-S3 Zero deja de hablar con la nube de Elato y pasa a usar un servidor local (Mac mini) que transcribe con faster-whisper, consulta al agente **Hermes** y sintetiza voz con Piper. Este runbook esta escrito para que **cualquier agente o persona** pueda ejecutarlo de principio a fin.

NOTA El codigo del puente ya existe en `server/hermes-bridge/bridge.py` y el firmware ya esta cambiado a `DEV_MODE` (commit correspondiente). Lo que queda es la instalacion en la Mac mini y ajustar la IP. El diseño y sus razones estan en `[[propuesta-hermes-bridge]]`.

Flujo de operacion final



Desde 2026-10-08 cada aparato tiene su propia clave y el bridge rechaza al que no la tenga (`BRIDGE_AUTH_MODE=enforce`); el endpoint `generate_auth_token` del puerto 3000 quedo apagado (404). Si Hermes tarda mas de 25 s, el parlante avisa y el resultado llega por Telegram. Todo eso esta en `[[claves-y-tareas-largas]]`.

El pipeline usa **LLM en modo streaming**: el puente empieza a sintetizar voz al recibir la primera frase completa del modelo (boundary en `.!?\n`), sin esperar la respuesta entera. La latencia percibida es mucho menor que en la version inicial.

Requisitos previos

- Mac mini con **Hermes instalado y funcionando** (`hermes_gateway` arranca).
- Homebrew y Python 3.10+ en la Mac mini.
- El repo ElatoAI clonado o copiado en la Mac mini (basta `server/hermes-bridge/`).
- El ESP32-S3 Zero con su montaje validado (ver `[[conexionado-por-componente]]`).
- Un equipo con PlatformIO para reflashear el firmware (paso 6).

Paso 1 — Habilitar la API de Hermes

En la Mac mini, editar `~/.hermes/.env` y añadir:

```
API_SERVER_ENABLED=true
API_SERVER_KEY=elige-una-clave-secreta
```

Reiniciar el gateway y comprobar que aparece la línea del API server:

```
hermes gateway
# esperado en el log: [API Server] API server listening on http://127.0.0.1:8642
```

Verificar con curl:

```
curl http://localhost:8642/v1/chat/completions \
-H "Authorization: Bearer elige-una-clave-secreta" \
-H "Content-Type: application/json" \
-d '{"model":"hermes-agent","messages":[{"role":"user","content":"hola"}]}'
```

ADVERTENCIA La API de Hermes da acceso a todo su toolset (incluida la terminal). **No cambiar** `API_SERVER_HOST` de su valor por defecto (`127.0.0.1`): el puente corre en la misma máquina y no necesita que Hermes escuche en la red.

Paso 2 — Fijar la IP de la Mac mini

El firmware apunta a la IP que le pases al compilar en la variable de entorno `HERMES_SERVER_IP` (ver Paso 6). Reservar esa IP para la Mac mini en el router (reserva DHCP) para que nunca cambie.

```
# En la Mac mini, para conocer la IP actual:
ipconfig getifaddr en0 || ipconfig getifaddr en1
```

Paso 3 — Instalar el puente

```
cd <repo>/server/hermes-bridge
brew install opus # libopus, requerido por opuslib
python3 -m venv .venv
.venv/bin/pip install -r requirements.txt
```

El directorio incluye `voice_history.py` (stub funcional). El puente arranca con el, aunque sin persistencia ni contexto Telegram. Ver seccion "VoiceHistory y contexto Telegram" mas abajo si se quiere la version completa.

Paso 4 — Descargar la voz de Piper (español)

```
mkdir -p ~/piper-voices && cd ~/piper-voices
curl -LO https://huggingface.co/rhasspy/piper-voices/resolve/main/es/es_ES/davefx/medium/es_ES-davefx-medium.onnx
curl -LO https://huggingface.co/rhasspy/piper-voices/resolve/main/es/es_ES/davefx/medium/es_ES-davefx-medium.onnx
```

TIP Cualquier voz de `rhasspy/piper-voices` sirve (hay varias en español). Si se usa otra, ajustar `PIPER_VOICE` en el paso 5. El `.onnx.json` es obligatorio: el puente lee de ahi el sample rate.

Paso 5 — Probar el puente manualmente

```
cd <repo>/server/hermes-bridge
HERMES_API_KEY=elige-una-clave-secreta \
PIPER_VOICE=~/.piper-voices/es_ES-davefx-medium.onnx \
.venv/bin/python3 bridge.py
```

Salida esperada (la primera vez tarda ~2 s: carga el modelo Whisper):

```
... Loading faster-whisper model 'base'...
... Whisper model loaded in 1.8s
... Bridge ready: ws://0.0.0.0:8000/ token http://0.0.0.0:3000/api/generate_auth_token
... Auth de dispositivos: modo enforce, 2 registrados en ~/.config/hermes-bridge/devices.json
... Tareas largas: aviso a los 25 s sin texto, resultado por `hermes send --to telegram`
... Hermes endpoint: http://127.0.0.1:8642/v1/chat/completions (model hermes-agent)
... Whisper model: base | Piper voice: /Users/.../es_ES-davefx-medium.onnx
```

Verificar los endpoints desde otra maquina de la LAN:

```
# Token compartido viejo: apagado en modo enforce
curl -s -o /dev/null -w "%{http_code}\n" "http://$HERMES_SERVER_IP:3000/api/generate_auth_token"
# esperado: 404 (en modo transition responde {"token": "elato-local-token"})

# Sin clave, el bridge rechaza
curl -s -o /dev/null -w "%{http_code}\n" -X POST "http://$HERMES_SERVER_IP:8000/voice?device=prueba"
# esperado: 401

# Health (nuevo en version de produccion)
curl http://$HERMES_SERVER_IP:3000/health
curl http://$HERMES_SERVER_IP:8000/health
# esperado: {"status": "ok"}
```

Log por turno de conversacion (nuevo en version de produccion):

```
User: hola como estas
Hermes: Bien, gracias. ¿En qué te puedo ayudar? | STT=0.4s first_token=1.2s chunks=2
```

Variables de configuracion (todas opcionales)

Variable	Default	Uso
<code>HERMES_API_KEY</code>	(vacía)	La <code>API_SERVER_KEY</code> de Hermes
<code>HERMES_URL</code>	<code>http://127.0.0.1:8642/v1/chat/completions</code>	Endpoint LLM
<code>HERMES_TIMEOUT_S</code>	<code>60</code>	Timeout máximo entre tokens del stream SSE
<code>WHISPER_MODEL</code>	<code>base</code>	Modelo STT (<code>base</code> = rápido, <code>small</code> / <code>medium</code> = más preciso)
<code>PIPER_VOICE</code>	<code>~/piper-voices/es_ES-davefx-medium.onnx</code>	Voz TTS
<code>BRIDGE_LANGUAGE</code>	<code>es</code>	Idioma para Whisper
<code>BRIDGE_SYSTEM_PROMPT</code>	(prompt en español)	Personalidad del asistente
<code>VAD_SILENCE_MS</code>	<code>800</code>	Silencio que cierra la frase
<code>VAD_AGGRESSIVENESS</code>	<code>1</code>	Sensibilidad del VAD (0-3; subir si corta frases a mitad)
<code>BRIDGE_AUTH_TOKEN</code>	<code>elato-local-token</code>	Clave compartida vieja: solo la entrega <code>generate_auth_token</code> en modo <code>transition</code> . No sirve para entrar
<code>BRIDGE_AUTH_MODE</code>	<code>transition</code>	<code>enforce</code> (vigente) rechaza a quien no tenga clave propia. Ver [[claves-y-tareas-largas]]
<code>BRIDGE_DEVICES_FILE</code>	<code>~/config/hermes-bridge/devices.json</code>	Hashes de las claves (<code>devices_admin.py</code>)
<code>BRIDGE_LONG_TASK_SILENCE_S</code>	<code>25</code>	Segundos sin texto de Hermes para avisar y mandar el resultado por Telegram
<code>BRIDGE_MAX_HISTORY</code>	<code>20</code>	Mensajes de historial que se mantienen en memoria
<code>BRIDGE_TELEGRAM_CONTEXT</code>	<code>5</code>	Mensajes de Telegram inyectados como contexto (requiere VoiceHistory completo)

Paso 6 — Firmware: IP y flasheo

El firmware ya esta en `DEV_MODE` + `VOICE_SERVER_DENO`. La IP de la Mac mini **no esta escrita en el codigo**: se inyecta al compilar desde la variable de entorno `HERMES_SERVER_IP` (lo hace `firmware-arduino/scripts/hermes_server_ip.py`, registrado como `extra_scripts` en `platformio.ini`).

```
export HERMES_SERVER_IP=192.168.1.100 # <- la IP reservada de tu Mac mini
```

Conviene dejarlo en el `~/.bashrc` (o `~/.zshrc`) del equipo que compila, asi no hay que acordarse en cada build. Si la IP cambia, se recompila con el nuevo valor: no hay que tocar `Config.cpp`.

AVISO: si `HERMES_SERVER_IP` no esta definida, el build **no falla** pero usa el placeholder `192.168.1.100` de `Config.cpp` y el firmware no va a encontrar el puente. El script lo avisa en la salida de `pio run`: `[hermes] AVISO: HERMES_SERVER_IP no esta definida...`

Compilar y flashear (el Zero necesita `--no-stub`, ya configurado; detalles en [[firmware-binarios-y-flasheo]]):

```
cd firmware-arduino
pio run -t upload --upload-port /dev/ttyACM0
```

NOTA (actualizada 2026-10-08): el bridge **si** valida la clave. Despues de flashear, crear la clave del aparato (`devices_admin.py add <nombre>`) y cargarla en `http://<IP>/wifi` → "Clave del bridge". Ojo: la "clave actual" puede ser el JWT viejo de la nube de Elato y no `elato-local-token` (el firmware solo pide clave si la NVS esta vacia). Procedimiento y salida en [[claves-y-tareas-largas]].

Despues de flashear por USB el Zero suele quedar en modo descarga: hay que **desenchufarlo y enchufarlo sin tocar BOOT** para que arranque.

Paso 7 — Verificacion end-to-end

1. Con el puente corriendo, encender el ESP32.
2. **Log del puente:** debe aparecer `Token request from MAC B8:F8:62:D7:59:C4` y luego `Device connected (MAC ...)`.
3. **LED:** verde/amarillo (escuchando) tras el arranque.
4. **Hablar** una frase corta y callar: el puente muestra `User: <texto>` y el altavoz responde antes de que Hermes termine (streaming). El log muestra `STT=Xs first_token=Xs chunks=N` al finalizar el turno.

5. El **boton KY-004** sigue funcionando igual (dormir/despertar).

Monitor serie del ESP32 (opcional): `[WSc] Connected to url: /`, luego `AUDIO.COMMITTED` / `RESPONSE.CREATED` / `RESPONSE.COMPLETE` en cada turno. `RESPONSE.CREATED` aparece varias veces por turno (una por frase sintetizada).

Paso 8 — Dejarlo como servicio (launchd)

```
cd <repo>/server/hermes-bridge
# 1. Editar com.elato.hermes-bridge.plist: rutas reales del venv/repo,
#   HERMES_API_KEY y PIPER_VOICE (buscar "AJUSTAR").
cp com.elato.hermes-bridge.plist ~/Library/LaunchAgents/
launchctl load ~/Library/LaunchAgents/com.elato.hermes-bridge.plist
# logs:
tail -f ~/Library/Logs/hermes-bridge.log
```

Con `KeepAlive` el puente se reinicia solo si se cae, y arranca al encender la Mac mini. El ESP32 reintenta la conexión WebSocket cada 1 s automáticamente, así que un reinicio del puente se recupera sin tocar la placa.

Para detenerlo limpiamente (el bridge registra la señal en el log):

```
launchctl stop com.elato.hermes-bridge # lo detiene; KeepAlive lo relanza
launchctl unload ~/Library/LaunchAgents/com.elato.hermes-bridge.plist # lo desactiva
```

VoiceHistory y contexto Telegram

El repo incluye `voice_history.py` como stub funcional: el puente arranca y conversa normalmente, pero sin persistencia ni integración con Telegram.

La implementación completa (disponible en la Mac mini de DebBot) añade: - Persistencia del historial de conversación por MAC del dispositivo. - `get_telegram_context()`: inyecta los últimos N mensajes del canal de Telegram al historial antes de consultar a Hermes, de modo que el asistente de voz sabe lo que se habló en ese canal.

Para activarla, sustituir `voice_history.py` por la versión completa de DebBot. La variable `BRIDGE_TELEGRAM_CONTEXT` controla cuántos mensajes se inyectan (default 5; poner 0 para desactivar aunque el módulo completo esté instalado).

Rollback a la nube de Elato

En `firmware-arduino/src/Config.h`, invertir los defines y reflashear:

```
// #define DEV_MODE
#define ELATO_MODE
...
// #define VOICE_SERVER_DENO
#define VOICE_SERVER_CLOUDFLARE
```

Nada mas: los certificados y URLs de Elato siguen intactos en `Config.cpp`. Tambien existe el respaldo `~/Firmware_ELATO_ok_limpio_merged.bin` (ver [\[\[firmware-binarios-y-flasheo\]\]](#)) que restaura la placa completa a la configuracion Elato con un solo comando.

Solucion de problemas

Sintoma	Causa probable	Solucion
<code>Piper voice not found</code> al arrancar	Falta la voz o la ruta es otra	Paso 4; ajustar <code>PIPER_VOICE</code>
<code>Hermes HTTP 401</code> en el log	<code>HERMES_API_KEY</code> no coincide	Usar la misma <code>API_SERVER_KEY</code> de <code>~/.hermes/.env</code>
<code>Hermes HTTP</code> con connection refused	Hermes no corre o API no habilitada	Paso 1; <code>hermes gateway</code> activo
El ESP32 no conecta y el log dice <code>[auth] ws: RECHAZADO</code>	Sin clave o con una que no esta registrada	Cargarla en <code>http://<IP>/wifi</code> ([[claves-y-tareas-largas]])
El portal dice "La clave actual no coincide"	La guardada no es la que crees (puede ser el JWT de Elato)	Ver la trampa en [[claves-y-tareas-largas]]
El Cardputer dice "Clave rechazada"	Clave mal tipeada, revocada o borrada	<code>devices_admin.py list</code> ; si se perdio, <code>delete</code> + <code>add --tecleable</code>
Telegram: "No se pudo completar: ..."	Hermes fallo (la causa va en el mensaje: filtro de DeepSeek, saldo, etc.)	Segun la causa; el detalle en <code>~/.hermes/logs/errors.log</code>
El ESP32 no conecta al bridge	IP mal puesta en el build o firewall	Verificar paso 6 y el firewall de macOS (permitir Python)
Conecta pero nunca transcribe	VAD no detecta voz o ruido de fondo alto	Bajar <code>VAD_AGGRESSIVENESS</code> a 0 o subir a 2; revisar el micro
Transcribe frases vacias	Doble-VAD (no deberia ocurrir con <code>vad_filter=False</code>)	Verificar que se usa la version de produccion del bridge
El altavoz corta a mitad de la primera frase	<code>VAD_SILENCE_MS</code> demasiado bajo	Subir <code>VAD_SILENCE_MS</code> a 1000
Audio entrecortado en el altavoz	CPU saturada; pacing roto	Cerrar cargas pesadas; usar <code>WHISPER_MODEL=base</code>
<code>first_token</code> muy alto (>5 s) en el log	Hermes usa herramientas (agentic)	Configurar <code>model_routes</code> en Hermes para respuestas directas
<code>ImportError: opuslib</code>	Falta libopus	<code>brew install opus</code>

Falta el archivo stub

Sintoma	Causa probable	Solucion
ModuleNotFoundError: voice_history		Verificar que <code>voice_history.py</code> esta en el mismo directorio que <code>bridge.py</code>
WS se desconecta cada ~60 s	heartbeat no configurado (version antigua)	Usar la version de produccion (<code>heartbeat=30.0</code>)
El puente no registra la señal de parada	Version antigua sin signal handlers	Usar la version de produccion

Referencia rapida

MAC MINI:

```
~/hermes/.env          -> API_SERVER_ENABLED=true, API_SERVER_KEY=...
server/hermes-bridge/  -> bridge.py + voice_history.py + requirements.txt + plist
~/piper-voices/        -> voz TTS (.onnx + .onnx.json)
puertos: 3000 (health; token apagado en enforce), 8000 (WS+voice+health), 8642 (Hermes, solo local)
claves: python3 devices_admin.py add|list|revoke|enable|delete (hashes en ~/.config/hermes-bridge/devices)
```

HEALTH CHECK:

```
curl http://$HERMES_SERVER_IP:3000/health -> {"status": "ok"}
curl http://$HERMES_SERVER_IP:8000/health -> {"status": "ok"}
```

FIRMWARE (firmware-arduino/):

```
src/Config.h          -> #define DEV_MODE + #define VOICE_SERVER_DENO
IP del puente          -> env HERMES_SERVER_IP (inyectada al compilar)
flasheo                -> pio run -t upload --upload-port /dev/ttyACM0
```

SERVICIO:

```
launchctl load ~/Library/LaunchAgents/com.elato.hermes-bridge.plist
launchctl stop com.elato.hermes-bridge # parada temporal (KeepAlive lo relanza)
launchctl unload ~/Library/LaunchAgents/com.elato.hermes-bridge.plist # desactivar
tail -f ~/Library/Logs/hermes-bridge.log
```

LOG POR TURNO:

```
[auth] ws: dispositivo 'esp32-parlante' (MAC, IP)
User: <texto>
Hermes: <respuesta> | STT=0.4s first_token=1.2s chunks=2
(tarea larga) Tarea larga: 25 s sin texto ... / [tarea larga] Telegram: enviado
```

ROLLBACK: Config.h -> ELATO_MODE + VOICE_SERVER_CLOUDFLARE y reflashear.